Swiggy Order Bot

AI-Powered Restaurant Ordering Assistant

Technical Documentation

Position: AI Full-Stack Developer

Assignment: AI Restaurant Order Automation

Stack: FastAPI + Next.js + Google Gemini + Telegram

Repository: github.com/Krishnareddy2468/Swiggybot

Table of Contents

1. Technical Overview
2. System Architecture
3. API Integration Approach
4. Core Features
5. Conversation Flow & State Machine
6. AI Intent Detection Pipeline
7. API Endpoints Reference
8. Project Structure
9. Setup & Testing Instructions
10. Challenges & Solutions

1. Technical Overview

This project is a conversational AI bot that automates food ordering from Swiggy. Users interact with the bot through natural language — either via a web-based chat interface or Telegram — and the bot handles the entire ordering workflow: searching restaurants, browsing menus, managing a cart, placing orders, and tracking delivery status.

Key Design Decisions

Decision	Choice	Rationale
Backend Framework	FastAPI (Python)	Native async support, automatic OpenAPI docs, Pydantic validation built-in
Frontend	Next.js (React)	Server-side rendering, fast HMR during development, component architecture
AI Engine	Google Gemini 2.0 Flash	Low latency (~500ms), strong at structured JSON extraction, free tier available
Chat Platform	Telegram Bot API	Well-documented, no approval needed (unlike WhatsApp Business API), rich formatting
Data Layer	In-memory + JSON file	Sufficient for prototype; sessions in-memory, orders persisted to disk
Food Platform	Mock Swiggy MCP	Swiggy has no public API; mock data simulates realistic restaurant/menu data

What the Bot Can Do

- Search restaurants by location, cuisine type, or name
- Display menus with numbered items, prices, veg/non-veg indicators, and bestseller tags
- Accept orders via item numbers, item names, or natural language ("add 2 margherita")
- Manage a shopping cart (add, remove, update quantity, view totals with GST)
- Complete checkout with delivery address and order confirmation
- Simulate order tracking with status progression and delivery partner assignment

- Fall back to keyword parsing when the AI API is unavailable

2. System Architecture

```
+-----------------+     +-----------------+
|   Telegram App   |     |  Web Chat (Next) |
+--------+---------+     +--------+---------+
         |                       |
Telegram Bot API            HTTP REST API
         |                       |
+--------v-----------------------v---------+
|              FastAPI Backend              |
|                (Port 8000)                |
+-------------------------------------------+
|                                           |
|   +------------+    +-----------------+   |
|   | Chat Route |    | Restaurant Route|   |
|   +-----+------+    +--------+--------+   |
|         |                    |            |
|   +-----v--------------------v--------+   |
|   |         Gemini AI Agent           |   |
|   |  (Intent Detection + NLU Engine)  |   |
|   +--+----------+----------+----------+   |
|      |          |          |              |
|  +---v---+ +----v----+ +---v----------+  |
|  |Session| |Order    | |Restaurant    |  |
|  |Service| |Service  | |Service       |  |
|  +---+---+ +----+----+ +---+----------+  |
|      |          |          |              |
+------+----------+----------+--------------+
       |          |          |
+------v---+ +----v----+ +---v--------------+
|In-Memory | |JSON File| |Mock Swiggy Data  |
|Sessions  | |Orders   | |10 Restaurants    |
+----------+ +---------+ +------------------+
```

Component Responsibilities

Component	Role
`main.py`	FastAPI app setup, CORS configuration, Telegram bot lifecycle
`gemini_agent.py`	Core NLU engine. Sends messages to Gemini with context, parses intent + entities, routes to handlers
`session_service.py`	Manages per-user state: conversation history, cart, selected restaurant, delivery address

`order_service.py`	Order placement, status tracking, delivery simulation with background tasks
`restaurant_service.py`	Restaurant search (fuzzy matching), menu retrieval, item lookup
`telegram_bot.py`	Telegram integration: command handlers, message routing, markdown formatting
`swiggy_data.py`	Mock data layer: 10 restaurants, 150+ items, Hyderabad locations

3. API Integration Approach

The assignment required integrating with Swiggy or Zomato via their MCP servers or public APIs. Since neither platform provides a publicly accessible API for third-party ordering, I implemented a **Mock Swiggy MCP Server** that simulates the data layer.

Mock Data Design

- **10 restaurants** with real brand names (Domino's, Pizza Hut, Meghana Foods, McDonald's, etc.)
- **150+ menu items** with realistic prices, descriptions, ratings, veg/non-veg flags
- **15 Hyderabad locations** (Madhapur, HITEC City, Gachibowli, Banjara Hills, etc.)
- **18 cuisine types** (Pizza, Biryani, Burgers, South Indian, Chinese, etc.)
- **Delivery simulation** with partner names, phone numbers, time estimates

Restaurant Service API

The `RestaurantService` class provides the same interface a real Swiggy API would:

Method	Equivalent Swiggy Endpoint
`search_restaurants(query, location, cuisine)`	GET /restaurants/search
`get_restaurant_by_id(id)`	GET /restaurants/{id}
`get_menu(restaurant_id)`	GET /restaurants/{id}/menu
`find_menu_item(restaurant_id, query)`	GET /menu/search
`get_bestsellers(restaurant_id)`	GET /restaurants/{id}/bestsellers

> If Swiggy were to provide API access, swapping the mock service with a real HTTP client would only require changing the `restaurant_service.py` implementation. The rest of the system (AI agent, session management, frontend) would work unchanged.

4. Core Features

Restaurant Discovery

Users can search restaurants by location ("restaurants in Madhapur"), by cuisine ("pizza places"), by name ("find Domino's"), or any combination. Results show ratings, delivery time, cost for two, and available offers.

Menu Browsing

Menus are displayed with numbered items organized by category. Users can filter for vegetarian items only. Each item shows its price, description, veg/non-veg flag, and bestseller badge if applicable.

Cart Management

Items can be added by menu number (type "3"), by name ("add margherita"), or with quantity ("2 garlic bread"). Adding the same item again increments the quantity. The cart shows itemized totals with 5% GST and delivery fee.

Order Flow

Checkout presents the order summary and asks for a delivery address. After address confirmation, the user reviews the final total and confirms. The system generates a unique order ID (e.g., SWG070575EAE2) and begins tracking simulation.

Order Tracking

Orders progress through: Confirmed → Being Prepared → Out for Delivery → Delivered. The web chat shows a "Refresh Status" button. On Telegram, the bot sends push notifications as the status changes.

5. Conversation State Machine

The bot uses a finite state machine to track where each user is in the ordering flow. This ensures context-appropriate responses — for example, typing a number while viewing a restaurant list selects a restaurant, but typing a number while viewing a menu adds that item to cart.

State	Description	Expected Input
IDLE	Initial state, no active context	Search query, greeting, help
SEARCHING	Restaurant results are shown	Restaurant number or name
BROWSING_MENU	A restaurant menu is displayed	Item number, item name, veg filter
ORDERING	Items have been added to cart	More items, checkout, show cart
AWAITING_ADDRESS	Cart is ready, need address	Delivery address text
CONFIRMING_ORDER	Order summary shown, awaiting yes/no	"Yes" to confirm, "No" to cancel
ORDER_PLACED	Order is placed and tracking	Track status, new order

6. AI Intent Detection Pipeline

Every user message goes through a two-stage processing pipeline:

Stage 1: Gemini AI Analysis

The message is sent to Google Gemini along with a system prompt containing:

- Current conversation state
- Selected restaurant and its menu (if applicable)
- Current cart contents
- Last 6 messages for conversation context

Gemini returns a JSON object with the extracted intent and entities:

```
{
  "intent": "add_to_cart",
  "entities": {
    "items": [{"name": "margherita pizza", "quantity": 2}]
  },
  "response_hint": "User wants to add 2 margherita pizzas"
}
```

Stage 2: Fallback Parser

If the Gemini API call fails (network error, rate limiting, invalid response), the system falls back to a keyword-based parser that uses:

- Word-level matching (not substring) for greetings and confirmations
- State-aware defaults (menu state → add to cart, search state → select restaurant)
- Location and cuisine keyword detection from the mock data

Supported Intents

Intent	Trigger Examples
greet	"hi", "hello", "hey"
search_restaurants	"show pizza places in Madhapur"
select_restaurant	"1", "2" (after seeing list)
show_menu	"menu", "show menu"
add_to_cart	"add margherita", "3" (menu number)
show_cart	"cart", "my order"
checkout	"checkout", "place order"
provide_address	Any text when in AWAITING_ADDRESS state
confirm_order	"yes", "confirm"
track_order	"track", "order status"

7. API Endpoints Reference

Method	Endpoint	Description
GET	/	Health check and service status
GET	/docs	Interactive Swagger documentation
POST	/api/chat/message	Send a message, receive bot response
GET	/api/chat/session/{user_id}	Retrieve session state
POST	/api/chat/reset/{user_id}	Reset user session
GET	/api/restaurants/	Search restaurants (query params: query, location, cuisine, veg_only)
GET	/api/restaurants/locations	List all available delivery locations
GET	/api/restaurants/cuisines	List all cuisine types
GET	/api/restaurants/{id}	Get full restaurant details with menu
GET	/api/restaurants/{id}/menu	Get menu with optional category/veg filter
GET	/api/restaurants/{id}/bestsellers	Get bestseller items

Chat Message Request/Response

```
POST /api/chat/message
Content-Type: application/json

{
  "message": "show me pizza places in madhapur",
  "user_id": "web_1234567890",
  "user_name": "Krishna"
}

Response:
{
  "response": "Here are the top restaurants...",
  "state": "searching",
  "cart_items": [],
  "restaurant": null,
```

```
  "order": null
}
```

8. Project Structure

```
Swiggybot/
├── backend/
│   ├── main.py                      # Entry point, CORS, lifecycle
│   ├── requirements.txt
│   └── app/
│       ├── models/schemas.py        # Pydantic: Order, CartItem, Session
│       ├── services/
│       │   ├── gemini_agent.py      # AI agent (NLU + handlers)
│       │   ├── session_service.py   # User session management
│       │   ├── order_service.py     # Orders + delivery simulation
│       │   ├── restaurant_service.py # Search + menu lookup
│       │   └── telegram_bot.py      # Telegram integration
│       ├── routes/
│       │   ├── chat.py              # /api/chat/* endpoints
│       │   └── restaurants.py       # /api/restaurants/* endpoints
│       └── mock_data/
│           └── swiggy_data.py       # 10 restaurants, 150+ items
│
├── frontend/
│   └── app/
│       ├── page.js                  # Chat UI component
│       ├── layout.js                # Root layout
│       └── globals.css              # Dark theme CSS
│
├── .env.example
├── .gitignore
└── README.md
```

9. Setup & Testing Instructions

Prerequisites

- Python 3.9 or higher
- Node.js 18 or higher
- Google Gemini API key (from aistudio.google.com)
- Telegram Bot Token (from @BotFather on Telegram)

Installation Steps

```
git clone https://github.com/Krishnareddy2468/Swiggybot.git
cd Swiggybot

# Set up environment variables
cp .env.example .env
# Edit .env with your GEMINI_API_KEY and TELEGRAM_BOT_TOKEN

# Backend
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python main.py              # Runs on http://localhost:8000

# Frontend (new terminal)
cd frontend
npm install
npm run dev                 # Runs on http://localhost:3000
```

Testing on Web

1. Open http://localhost:3000 in your browser
2. Click a suggestion card or type "Show me restaurants in Madhapur"
3. Select a restaurant by typing its number
4. Add items by number (e.g., "3") or by name (e.g., "add chicken biryani")
5. Type "checkout" and follow the prompts
6. After placing an order, click "Refresh Status" to track progress

Testing on Telegram

1. Search for your bot on Telegram (the username you set with @BotFather)
2. Send /start to begin the conversation
3. Use the same natural language as the web chat

10. Challenges & Solutions

Challenge 1: Substring matching in fallback parser

The keyword-based intent parser used substring matching, causing "hi" to match inside words like "Shahi" (as in "Shahi Gulab Jamun"). This incorrectly triggered a greeting response when the user was trying to order a menu item.

Solution: Replaced substring matching with word-level set intersection. Also restructured the fallback logic to check the conversation state first — when a user is browsing a menu, any unrecognized input defaults to an add-to-cart action rather than falling through to greeting detection.

Challenge 2: Session persistence during development

Uvicorn's auto-reload feature (useful during development) clears all in-memory sessions when backend files change. This caused cart data to disappear mid-conversation.

Solution: Stored the user ID in the browser's sessionStorage so page reloads don't generate new session IDs. For production, sessions would be backed by Redis or a database.

Challenge 3: Gemini response format inconsistency

Gemini occasionally wraps its JSON output in markdown code blocks, causing JSON parse errors.

Solution: Added a preprocessing step that strips markdown fencing (```json ... ```) before parsing. If parsing still fails, the system gracefully falls back to the keyword-based parser.

Challenge 4: Telegram message length limits

Menus with 20+ items exceed Telegram's 4096-character message limit.

Solution: Messages are automatically split into chunks under 4000 characters. Markdown formatting is attempted first, with a plaintext fallback if Telegram rejects the markdown syntax.